/tikz/,/tikz/graphs/
conversions/canvas coordinate/.code=1 , conversions/coordinate/.code=1

trees

CS 182 Final Exam

2025-12-15 16:39:11-08:00

3. CNN Basics (10 pts)

We are going to describe a convolutional neural net using the following pieces:

- CONV(K, F, P, S) denotes a convolutional layer with F different filters, each of size $K \times K \times C$, where C is the depth (i.e. number of channels) of the activations from the previous layer. Padding is P, and stride is S.
- POOL(2) denotes a 2×2 max-pooling layer with stride 2 (pad 0)
- FLAT just turns whatever shape input tensor into a one-dimensional array with the same values in it.
- LIN(D) denotes a single fully-connected linear layer with D output activations. The number of input activations implicitly comes from the previous layer.

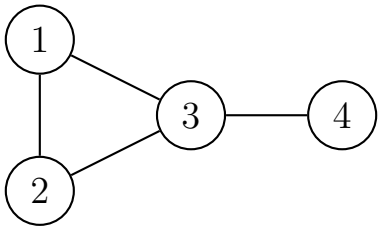
Note: All CONV and LIN layers have biases as well as weights. Do not forget the biases when counting parameters.

Now, we are going to use this network to do inference on a single input. **Fill in the missing entries in this table of the size of the number of parameters at each layer and the shape of the output activations at each layer.** You can/should write your answer as a computation (e.g. $128 \times 128 \times 3$) in the style of the already filled-in entries of the table. You don't have to evaluate it, but that can help check your work at the end.

Layer	**Number of Parameters**	**Dimension of Activations in output of layer**
Input	0	$26 \times 26 \times 1$
CONV$(5, 10, 1, 1)$	$5 \times 5 \times 1 \times 10 + 10$	$24 \times 24 \times 10$
POOL(2)	0	$12 \times 12 \times 10$
CONV$(3, 20, 1, 1)$		
CONV$(1, 20, 0, 2)$		
FLAT	0	720
LIN(10)		10

4. DiffPool (12 pts)

Consider the following graph with 4 nodes:



Suppose each node has 2-dimensional features given by feature matrix X where row i of X is the feature vector for node i:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 0 \end{bmatrix}$$

You will perform **one DiffPool layer**:

$$4 \text{ nodes} \rightarrow 2 \text{ clusters}$$

Suppose we additionally have GNNs which give the following (unnormalized) scores (in S) for each node and embeddings (in Z):

$$S = \begin{bmatrix} 2 & -\infty \\ 3 & -\infty \\ 1 & 1 \\ 0 & \ln 3 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \\ 2 & 1 \end{bmatrix}$$

(a) (6pts) Compute the node embeddings at the output of the diffpool:

$$X^{(1)} = (S_{\text{softmax}})^T Z.$$

Remember that $S_{softmax}$ takes the cluster scores for each node and turns them into a probability via the softmax operation (which takes the exponential of scores and then normalizes all those positive numbers into a probability).

(b) (6pts) Recall that to make training more effective, DiffPool adds auxiliary losses.

For each of the following auxiliary losses, **select the primary design purpose of using the auxiliary loss**.

i. LP loss: $\|A - S_{\text{softmax}} S_{\text{softmax}}^T\|_F^2$ where A is the adjacency matrix of the graph.

◯ Align clusters to graph topology

◯ Encourage hard assignments of nodes to clusters

ii. Average entropy minimization loss: $-\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} S_{\text{softmax}}[i,j] \log S_{\text{softmax}}[i,j]$

◯ Align clusters to graph topology

◯ Encourage hard assignments of nodes to clusters

5. Muon and Other Optimizers (14 pts)

In this question, we will focus on an idealized scenario where we are training a single linear layer with weight matrix W, and our batch consists entirely of copies of two datapoints whose individual gradients $\nabla_W L$ are:

$$\text{Gradient wrt. W for datapoint A : } \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \text{Gradient wrt. W for datapoint B : } \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \tag{1}$$

Assume that the training batch consists entirely of a copies of datapoint A and b copies of datapoint B. The overall full-batch gradient is now:

$$\text{Gradient : } \begin{bmatrix} a+b & a-b \\ a-b & a+b \end{bmatrix} \quad \text{which has SVD:} \quad \left(\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\right)\begin{bmatrix} 2a & 0 \\ 0 & 2b \end{bmatrix}\left(\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}\right). \tag{2}$$

(a) (6pts) Assume $a = 1, b = 2$. What is the resulting update ΔW when the gradient is used by gradient descent, SignGD, and Muon respectively? Assume a learning rate of 1 and no momentum. Assume an idealized Muon that takes the gradient and sets all its singular values to 1 instead of iterating through a matrix polynomial.

GD : _____

SignGD : _____

Muon : _____

(b) (4pts) You increase your batch by inserting more copies of A and B at the same 1 : 2 ratio. **Which of the three updates above change from part (a)?**

☐ GD

☐ SignGD

☐ Muon

(c) (4pts) You now have a differently imbalanced training batch where $a = 10, b = 1$. Which of the three updates above change from part (a)?

☐ GD

☐ SignGD

☐ Muon

6. Quantization Aware Training (14 pts)

Recall from homework that linear quantization is defined by the goal of

$$r \approx S(q - Z) \tag{3}$$

where r is a floating point real number, q is an n-bit integer, Z is an n-bit integer, and S is a floating point real number. Z is the quantization zero point and S is quantization scaling factor. Both Z and S are quantization parameters that can be kept at higher precision because their cost will be amortized over lots of entries in a weight matrix being quantized together.

The actual quantization is done using rounding to the nearest integer:

$$q = \text{round}\left(\frac{r}{S} + Z\right). \tag{4}$$

Consider a 3×3 weight matrix whose values are represented in standard FP32 (float) format as

$$\begin{bmatrix} 0.25 & 2.0 & 1.5 \\ -1.0 & 0.5 & 0.25 \\ 0.4 & -1.0 & 0.5 \end{bmatrix}. \tag{5}$$

Now, you would like to use linear quantization to represent this weight matrix using 3 bits for q, so $q_{\min} = -4$ and $q_{\max} = 3$, and the eight possible values for q are: -4,-3,-2,-1,0,1,2,3. Let $r_{\min} = -1.0$ and $r_{\max} = 2.0$ be the smallest and largest values in the weight matrix. In order to use the full dynamic range of q, we should correspond $q_{\min}$ to $r_{\min}$ and $q_{\max}$ to $r_{\max}$.

(a) (9pts) What does the weight matrix (5) look like after applying linear quantization using the best computed S and Z values? Carefully look at the partially filled entries and **complete the empty entries in the Q matrix below:**

$$Q = \begin{bmatrix} \square & \square & 2 \\ -4 & -1 & -1 \\ \square & -4 & -1 \end{bmatrix} \tag{6}$$

(Hint: While you can solve for the blanks using explicit computation of the S and Z and applying (4), you can also get the values by inspection and thinking about what must happen in quantization.)

(b) (5pts) Quantization-aware training quantizes the full-precision model weights w to w_q during each forward pass of training. It uses these quantized weights for computing activations. For backpropagation, we apply the "straight-through" estimator to compute the backpropped gradient through the quantization step to the underlying full-precision weights. This straight-through estimator treats the effect of quantization as though it were random external noise added to the full-precision weights. **Why do we do this?** (Select all that apply)

- ☐ Quantization is either nondifferentiable (when viewed as a function into the integers) or has zero derivatives almost everywhere (when viewed as a function from $\mathbb{R} \to \mathbb{R}$) and would prevent meaningful updates if we actually differentiated it.

☐ Because quantization saves memory during training when storing the momentum terms in the optimizer.

☐ Because we want the final model to be used after quantization for inference, it is useful to allow training to learn how to reduce the loss due to quantization.

7. Your Language Model is Secretly a Reward Model (24 pts)

In homework, you worked through the foundations of Direct Preference Optimization (DPO). If we optimize our policy $\pi_\theta(y|x)$ to maximize our reward model $r(x, y)$ (that tells us quantitatively how good y is as a response to x) while trying to stay close to a reference policy $\pi_{\text{ref}}(y|x)$ in KL Divergence:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)}[r(x, y)] - \beta D_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$$

Then, the optimal solution takes the form:

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right) \tag{7}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right)$ normalizes the probability and β is the hyperparameter that determines how much we want to stay close to the reference distribution π_{ref}. Here, x is the prompt/question and y is the response from the model.

(a) (6pts) Manipulate (7) **to express** $r(x, y)$ **in terms of** $\pi^*(y|x)$**,** $\pi_{\text{ref}}(y|x)$**,** β**, and** $Z(x)$**.**

(b) (4pts) Looking at your expression in the previous part, is knowledge of $Z(x)$ **required to understand if the implicit reward prefers response** y_1 **to** y_2 **in response to prompt/question** x**?**

◯ Yes, we need $Z(x)$

◯ No, we do not need $Z(x)$

(c) (8pts) Now, consider the DPO gradient you derived in homework for training a network using a dataset consisting of triplets: two responses for each prompt x. The response that humans prefer is y_w and the response that humans don't prefer is y_l.

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\beta\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{weight}}\underbrace{[\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x)]}_{\text{update direction}}\right]$$

where σ is the sigmoid function $\sigma(z) = 1/(1 + e^{-z})$, $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the implicit reward, and the distribution $\mathcal{D}$ just represents what SGD is doing.

(i) Consider a case where the model is extremely confident that the dispreferred response y_l is actually better than the preferred response y_w (i.e., $\hat{r}_\theta(x, y_l) \gg \hat{r}_\theta(x, y_w)$). **What**

happens to the weight term in the gradient, i.e. $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$**?** Select one response.

◯ The gradient weight becomes substantial, approaching 1 if $\hat{r}_\theta(x, y_l) \gg \hat{r}_\theta(x, y_w)$.

◯ The gradient weight approaches zero if $\hat{r}_\theta(x, y_l) \gg \hat{r}_\theta(x, y_w)$.

◯ The gradient weight approaches 1/2 if $\hat{r}_\theta(x, y_l) \gg \hat{r}_\theta(x, y_w)$.

(ii) Why is the adaptive-weighting in DPO desirable compared to the standard supervised fine-tuning (SFT) approach that simply maximizes $\mathbb{E}_{(x,y_w)\sim\mathcal{D}} \log(\pi_\theta(y_w|x))$? (**Mark all that apply**)

☐ Our preference training data just tells us which of two responses were better. It doesn't tell us if both were somewhat bad or both were good but one was better. And yet, standard SFT would treat all the preferred responses as though they were gold-standard perfect responses and completely ignore the information present in knowing which alternatives are less preferred.

☐ Because standard SFT would apply the same amount of update towards winners even when they were already winning, it is more prone to triggering catastrophic forgetting in the model during a fine-tune, especially as we fine-tune for multiple epochs. The DPO updates are comparatively more minimal.

☐ Attenuating the strength of the gradients in this DPO way helps prevent numerical overflow as compared to standard SFT.

☐ Doing the DPO update better prevents vanishing gradients.

(d) (6pts) Imagine you are full fine-tuning a 7B instruction-following model using DPO with β selected from the low end of the typical range and reference policy $\pi_{\text{ref}} = \pi_{\text{SFT}}$. After several epochs of training on preference pairs, you observe:

- Training loss: decreased significantly from initial values
- KL divergence $D_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$: has grown substantially larger than typically recommended
- Generated responses: contain repetitive phrases and degenerate patterns. They have gotten worse than what π_{SFT} generated.
- Held-out preference accuracy: very high (model correctly ranks test preferences better than π_{SFT} did)
- Human evaluation on new prompts: responses are often incoherent or low-quality

Below are potential strategies for addressing the training issues identified above. **Select ALL strategies that would likely be helpful.**

- ☐ Increase β substantially (e.g., by a factor of 3-5x) to strengthen the KL penalty and stay closer to the reference policy.
- ☐ Decrease β to allow the model more freedom to optimize the preference objective.
- ☐ Implement early stopping.
- ☐ Continue training for additional epochs to allow the model to find better solutions.
- ☐ Use DPO with a LoRA fine-tune that trains fewer learnable parameters instead of a full fine-tune.

8. Using Redundancy in Parameterization (15 pts)

Assume you are given the following dense layer:

$$y = Wx + b \tag{8}$$

with the shape of the weights W being $d_{\text{out}} \times d_{\text{in}}$ and the biases b being d_{out}-dimensional. There is an iid random Gaussian initialization of:

$$W[i,j] \sim N(0, \frac{1}{d_{\text{in}}}) \quad b[i] \sim N(0, 10^{-4}) \tag{9}$$

for i, j ranging over the shape of the weight matrix and bias vector. Here, the first argument to N is the mean and the second argument is the variance.

You are dealing with a particularly annoying codebase in which you cannot adjust the Adam optimizer being used or change any of the optimizer parameters, including the learning rate, momentum terms, etc. Feel free to think about the SignGD optimizer as standing-in for Adam in this problem.

However, you are able to change the network definition in (8) as well as the initialization described by (9). **How can you adjust these definitions in (8) and (9) so that the input-output behavior training trajectory matches what would happen if you had used a 10x higher learning rate for Adam in the original network (8) and initialization (9)?**

$y =$ _____

$W[i,j] \sim$ _____

$b[i] \sim$ _____

9. Debugging Generative Models (12 pts)

You are training your first generative model, and you're very excited as your training and validation losses have both dropped to nearly zero. But when you try to generate some samples, only garbage comes out! What is wrong or missing in each of the following implementations, and/or how would you fix it? **Answer briefly. You don't need to give the exact fix in code, just tell us what you need to do.** If you say X is missing, we will understand that the implied fix is to include X.

(a) (6pts)

```
class CausalAttention(nn.Module):
    def forward(self, x):
        Q = self.W_q(x)  # (batch_size, seq_len, d_model)
        K = self.W_k(x)  # (batch_size, seq_len, d_model)
        V = self.W_v(x)  # (batch_size, seq_len, d_model)
        scores = torch.matmul(Q, K.transpose(-2, -1))
        scores = scores / math.sqrt(self.d_model)
        attention_weights = F.softmax(scores, dim=-1)
        attn_output = torch.matmul(attention_weights, V)
        output = self.W_o(attn_output)
        return output
```

Above, we have what should be a causal attention block used as part of a standard Transformer language model recipe for a GPT-style model pretrained using batches consisting of teacher-forced entire sequences and next-token prediction loss. To sample from the model, we would generate tokens autoregressively. **What is wrong with the above implementation for training?**

(b) (6pts)

```
class VariationalAutoEncoder(nn.Module):
    def get_loss(self, x):
        # Encode x
        z_mean, z_log_std = self.encoder(x)

        # Add noise to z
        noise = torch.randn(z_mean.shape)
        z = z_mean + torch.exp(z_log_std) * noise

        # Decode z back to x, with mean-squared reconstruction loss.
        x_pred = self.decoder(z)
        loss = F.mse_loss(x, x_pred)
        return loss
```

Above, we have what should be the entire loss function for training a variational auto-encoder. To sample from this model, we would sample $z \sim N(0, 1)$, and decode with our learned decoder. **What is wrong with the above implementation for training?**

10. Rank of Updates (Problem context)

Throughout this problem, assume a learning rate of $\eta = 1$ and simple SGD with no momentum is being used to do weight updates.

(a) (6pts) Let $y = Wx$ represent a fully connected layer where $W \in \mathbb{R}^{d_{out} \times d_{in}}$, $x \in \mathbb{R}^{d_{in}}$ is the input to the layer, and $y \in \mathbb{R}^{d_{out}}$ is the output of the layer.

This layer is being trained with a minibatch consisting of a single input, and backprop from the final loss brings a nonzero upstream gradient $g = \nabla_y L$ where L is the final loss in our deep network. Note that g has the same shape as y since it is a gradient.

Recall that: $\frac{\partial L}{\partial W[i,j]} = x[j]\frac{\partial L}{\partial y[i]}$ for $y = Wx$.

What is the SGD update of the weight matrix, ie ΔW? (Your answer should be in terms of x, g.)

$$\Delta W = _____$$

$$\text{rank}(\Delta W) = 1$$

(b) (6pts) Now, assume we process a batch of B training samples at a time. That is, our input is now a matrix $X = [x_1, x_2, \ldots, x_B]$ with shape $d_{in} \times B$ and our upstream gradient (from backprop) is now a corresponding matrix G with shape $d_{out} \times B$ where the i-th column is $\nabla_{y_i} L_i$ the gradient of the loss with respect to the layer's output with input x_i. Assume that $B < \min(d_{in}, d_{out})$. **What is the new update of the weight matrix ΔW?**

$$\Delta W = _____$$

What is the rank of this update? Assume the inputs in the batch are linearly independent and the upstream gradient also has full rank.

$$\text{rank}(\Delta W) = _____$$

(c) (8pts) We will now consider learnable bias vectors. Consider a dense layer defined as $y = Wx + b$ where $W \in \mathbb{R}^{d_{out} \times d_{in}}$ and $b \in \mathbb{R}^{d_{out}}$. Again, assume that x has RMSNorm 1.

i. Suppose we did *not* use muP scaling — i.e. we use the same learning rate for all parameters, regardless of shape — **how much does an update to the bias vector Δb contribute to the corresponding expected RMS-norm-squared of the change in y, in comparison to the update to the weight matrix ΔW?** Express your answer as a rough proportion contribution-from-bias : contribution-from-dense-weight-matrix.

(Hint: For simplicity, think of the input x as being a vector entirely of 1s and the gradient $\nabla_y L$ coming from upstream as also a vector consisting entirely of 1s.)

Select one:

◯ $1 : 1$ i.e. same contribution

◯ $1 : 2$ i.e. Bias contributes half as much

◯ $1 : d_{in}$

◯ $1 : d_{out}$

◯ $1 : d_{in}^2$

◯ $1 : d_{out}^2$

◯ $d_{in} : 1$

◯ $d_{out} : 1$

◯ $d_{in}^2 : 1$

◯ $d_{out}^2 : 1$

ii. One option for applying muP to learnable biases is to treat biases as matrices with a shape of $d_{out} \times 1$, i.e. thinking of their $d_{in} = 1$ as though they were weights multiplying a constant feature whose value is always 1. We can then apply the same muP learning-rate scaling rule derived in part (a) of this problem for the learnable biases, along with applying the muP scaling for the weights based on their shape. **What is the resulting rough proportion of contribution-from-bias : contribution-from-dense-weight-matrix for the resulting updates?**

Below, d_{in} refers to the d_{in} of the learnable dense weight matrix. The d_{out} is the same for both the biases and the weight matrix.

Select one:

◯ $1 : 1$ i.e. same contribution

◯ $1 : 2$ i.e. Bias contributes half as much

◯ $1 : d_{in}$

◯ $1 : d_{out}$

◯ $1 : d_{in}^2$

◯ $1 : d_{out}^2$

◯ $d_{in} : 1$

◯ $d_{out} : 1$

◯ $d_{in}^2 : 1$

◯ $d_{out}^2 : 1$

(c) (6pts) Now, consider a convolutional layer with C_{in} input channels and C_{out} output channels, where each convolutional filter has spatial size $K \times K$. We will use same padding and a stride of 1 so that if the layer's input has size $W \times H \times C_{\text{in}}$, then the layer's output has size $W \times H \times C_{\text{out}}$.

Recall that such a convolutional layer can be represented as a linear layer, wherein all the filters are flattened together into a single matrix F and the input image is reshaped/recast into a matrix X so that each column of X corresponds to all the C_{in} input channels in a flattened $K \times K$ patch of the input image. Fill in the following blanks:

i. The reshaped filter F has matrix shape $C_{\text{out}} \times \underline{K \cdot K \cdot C_{\text{in}}}$.

ii. The output $Y = FX$ of this operation has shape _____ $\times \underline{W \cdot H}$.

(Hint: The output can be reshaped into an image of shape $W \times H \times C_{out}$. Make sure your answer is consistent!)

iii. To be able to properly represent this convolutional layer, the input X should have shape:

_____ $\times$ _____.

(d) (6pts) Building on the previous part where $Y = FX$, assume we are updating the weights of this convolutional layer after seeing a single training image. **What is the update of the (flattened) weight matrix ΔF in terms of X, which is the reshaped input image, and upstream gradient $G = \nabla_Y L$?** (The upstream gradient has also been shaped to match the operation output Y above.)

$\Delta F =$ _____

Even though we have only a single training image, **how big can the rank of the update matrix ΔF be?** Your answer should be in terms of $K, C_{\text{in}}, C_{\text{out}}, W, H$.

$\text{rank}(\Delta F) \leq$ _____

Briefly explain why this rank can be strictly bigger than 1 while in part (a), the rank is 1 when there's a single training example in the batch.

11. muP (24 pts)

The maximal update parameterization (muP) provides a helpful scaling law for learning rates, such that training dynamics remain similar as we adjust the size of our neural network. Throughout this problem, assume we are utilizing an off-the-shelf SignGD optimizer (standing in as an idealization of Adam) with a minibatch size of 1.

(a) (8pts) Consider a bias-free dense linear layer defined as $y = Wx$, where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Assume x has RMS Norm 1. **How should we scale the learning rate for W so that the RMS-norm change in y remains constant regardless of the shape of W?**

(Hint: Remember part (a) of Problem 10: "Rank of Updates" to help.)

Select one:

- ◯ $O(1)$ i.e. constant
- ◯ $O(\frac{1}{\sqrt{d_{\text{in}}}})$
- ◯ $O(\frac{1}{d_{\text{in}}})$
- ◯ $O(\sqrt{d_{\text{in}}})$
- ◯ $O(d_{\text{in}})$
- ◯ $O(d_{\text{in}} d_{\text{out}})$
- ◯ $O(\frac{1}{\sqrt{d_{\text{out}}}})$
- ◯ $O(\frac{1}{d_{\text{out}}})$
- ◯ $O(\sqrt{d_{\text{out}}})$
- ◯ $O(d_{\text{out}})$
- ◯ $O(\frac{d_{\text{in}}}{d_{\text{out}}})$
- ◯ $O(\sqrt{\frac{2}{d_{\text{in}}+d_{\text{out}}}})$
- ◯ $O(\frac{2}{d_{\text{in}}+d_{\text{out}}})$
- ◯ $O(\sqrt{\frac{d_{\text{in}}+d_{\text{out}}}{2}})$
- ◯ $O(\frac{d_{\text{in}}+d_{\text{out}}}{2})$

(b) (8pts) Let us now consider embedding layers of a language model, parameterized by a matrix $E \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Recall that performing lookup on an embedding layer can be understood as multiplying a one-hot vector of dimension d_{in}. **How should we scale the learning rate for the embedding layer in terms of its shape to ensure that the RMS-norm change in the output of the embedding layer remains constant regardless of d_{in} and d_{out}?**

Select one:

◯ $O(1)$ i.e. constant

◯ $O(\frac{1}{\sqrt{d_{\text{in}}}})$

◯ $O(\frac{1}{d_{\text{in}}})$

◯ $O(\sqrt{d_{\text{in}}})$

◯ $O(d_{\text{in}})$

◯ $O(d_{\text{in}}d_{\text{out}})$

◯ $O(\frac{1}{\sqrt{d_{\text{out}}}})$

◯ $O(\frac{1}{d_{\text{out}}})$

◯ $O(\sqrt{d_{\text{out}}})$

◯ $O(d_{\text{out}})$

◯ $O(\frac{d_{\text{in}}}{d_{\text{out}}})$

◯ $O(\sqrt{\frac{2}{d_{\text{in}}+d_{\text{out}}}})$

◯ $O(\frac{2}{d_{\text{in}}+d_{\text{out}}})$

◯ $O(\sqrt{\frac{d_{\text{in}}+d_{\text{out}}}{2}})$

◯ $O(\frac{d_{\text{in}}+d_{\text{out}}}{2})$

(c) (8pts) We will now consider learnable bias vectors. Consider a dense layer defined as $y = Wx + b$ where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and $b \in \mathbb{R}^{d_{\text{out}}}$. Again, assume that x has RMSNorm 1.

i. Suppose we did not use muP scaling — i.e. we use the same learning rate for all parameters, regardless of shape — how much does an update to the bias vector Δb contribute to the corresponding expected RMS-norm-squared of the change in y, in comparison to the update to the weight matrix ΔW? Express your answer as a rough proportion contribution-from-bias : contribution-from-dense-weight-matrix.

(Hint: For simplicity, think of the input x as being a vector entirely of 1s and the gradient $\nabla_y L$ coming from upstream as also a vector consisting entirely of 1s.)

Select one:

◯ $1 : 1$ (same contribution)

◯ $1 : d_{\text{in}}$

◯ $1 : d_{\text{in}}^2$

◯ $d_{\text{in}} : 1$

◯ $d_{\text{in}}^2 : 1$

◯ $1 : 2$ (Bias contributes half as much)

◯ $1 : d_{\text{out}}$

◯ $1 : d_{\text{out}}^2$

◯ $d_{\text{out}} : 1$

◯ $d_{\text{out}}^2 : 1$

ii. One option for applying muP to learnable biases is to treat biases as matrices with a shape of $d_{\text{out}} \times 1$, i.e. thinking of their $d_{\text{in}} = 1$ as though they were weights multiplying a constant feature whose value is always 1. We can then apply the same muP learning-rate scaling rule derived in part (a) of this problem for the learnable biases, along with applying the muP scaling for the weights based on their shape. What is the resulting rough proportion of contribution-from-bias : contribution-from-dense-weight-matrix for the resulting updates?

Below, d_{in} refers to the d_{in} of the learnable dense weight matrix. The d_{out} is the same for both the biases and the weight matrix.

Select one:

◯ $1 : 1$ (same contribution)

◯ $1 : d_{\text{in}}$

◯ $1 : d_{\text{in}}^2$

◯ $d_{\text{in}} : 1$

◯ $d_{\text{in}}^2 : 1$

◯ $1 : 2$ (Bias contributes half as much)

◯ $1 : d_{\text{out}}$

◯ $1 : d_{\text{out}}^2$

◯ $d_{\text{out}} : 1$

◯ $d_{\text{out}}^2 : 1$

12. Comparing Sequence Model Architectures: Compute (20 pts)

In this question, you will compare three major sequence model architectures: classic Recurrent Neural Networks (RNNs), Transformers, and modern State Space Models (SSMs). Let T be the sequence length and d be the embedding dimension. For State Space Models, let N be the state-expansion — i.e. by what factor (typically a number like 64) the state is bigger than the embedding dimension.

Assume that the cost of a single floating-point operation (FLOP) is constant. **Fill in the missing cells of the below table of runtimes (in big-O notation), and provide a brief explanation in the corresponding section on the next page.** You can assume that $T \gg d \gg N$.

Task	**RNNs**	**Transformers**	**SSMs (S4D style)**
Training cost	$O(Td^2)$ due to both RNN forward pass and backprop using T recurrent updates using matrices with dimension d	**(a)**	**(b)**
Training parallelization	Poor due to inherently sequential computation due to the nonlinearities encountered along the time axis	Excellent (every token's attention computation can be fully-independent)	Good (FFT can be parallel with depth $\log T$, but not as good as self-attention because of how GPUs are optimized.)
Inference cost for auto-regressive generation of a sequence of T tokens	**(c)**	**(d)**	$O(Td(d+N))$ for T steps of cost $O(d^2 + dN)$ due to an update of the state with dimension dN that breaks down into $O(d^2)$ to compute inputs/outputs and $O(dN)$ to do the independent scalar state updates. Under our assumptions, the cost simplifies to $O(Td^2)$.

Because this is a cheat-sheet free exam, we provide you with some S4D basics here. S4D is the style you saw in lecture before Mamba-style models with selective state spaces.

- In modern SSMs, the state-space block performs a role analogous to that of a causal multi-head-attention block in Transformer architectures.
- In S4D models, the state-space part has diagonal learnable dynamics and the operation of the state-space block is to perform a different learnable linear-time-invariant scalar convolution for each embedding dimension.
- It is possible to speed up the computation of a full-length convolution of a scalar sequence of length n with another scalar sequence of length n to happen in $O(n \log n)$ time using FFT-based tricks.
- It is possible to directly materialize a single scalar impulse response of length T using $O(NT)$ computation but only $O(N + T)$ memory.
- As is the case in multi-head-attention for computing the queries, keys, and values, $O(d^2T)$ computations are needed to compute all the inputs and outputs to S4D's core state-space-model. The counterparts to fully-connected feedforward MLPs also cost the same.

13. Residual Connections with PreNorms (20 pts)

The deeper a neural network grows, the more likely it will suffer from either vanishing gradients or exploding gradients due to the multiplicative nature of the chain rule.

Fig 1 shows a simplified example of a residual block with layer normalization in the now default "pre-norm" style.

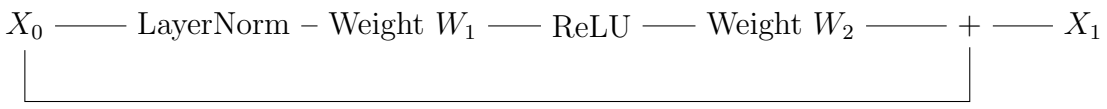


Figure 1: Diagram for a Residual Block with LayerNorm

(a) (3pts) Assuming that we have zero biases, **what is the expression for X_1 in terms of W_1, W_2 and X_0?** Assume $W_i \in \mathbb{R}^{n \times n}, X_i \in \mathbb{R}^n$. You can just call both LayerNorm and component-wise ReLU as functions here, you don't need to expand them.

(b) (5pts) Compute the Jacobian $\frac{\partial X_1}{\partial X_0}$. Express your answer in terms of W_1, W_2, the Jacobian of LayerNorm, J, and the Jacobian of the ReLU given by diagonal matrix D whose i-th term on the diagonal is 1 if the corresponding ReLU is on, and 0 if it is off.

Based on the structure of your result, explain why the residual connection helps prevent vanishing gradients during backprop for a deep network.

(c) (12pts) For simplicity, instead of LayerNorm, let us consider an RMSNorm layer with no learnable parameters that scales its input vector so that the output vector points in the same direction, but has RMSNorm 1.

If the input is the d-dimensional vector X, the output vector $Z = \sqrt{d}\frac{X}{\|X\|_2}$.

When backpropping the gradient through the RMSNorm Layer, what is the applied Jacobian $J = \frac{\partial Z}{\partial X}$?

(Hint: while it is completely possible to grind the calculation out using vector calculus, it might be useful to think about this Jacobian J using what the derivative means conceptually. Let $R(X)$ be the RMSNorm Layer acting on X. Then $R(X + \Delta X) \approx R(X) + J\Delta X + O(\|\Delta X\|^2)$.

To understand J, consider a neighborhood of X using a local orthogonal coordinate system. In other words, use a convenient coordinate system for ΔX. Let the first basis vector be aligned to X itself and have the rest be h that are orthogonal to it. Use the linearity of the derivative to understand those different directions separately.

What is the impact on the output of an RMSNorm Layer of growing/shrinking X by adding something aligned to X?

What is the impact on the output of an RMSNorm Layer of adding a tiny h that is orthogonal to X? — You might find it helpful to think about the Pythagorean theorem and the fact that $\sqrt{1+\Delta^2} \approx 1 + \frac{\Delta^2}{2}$ when Δ is small.

Once you understand what the matrix J does to all basis vectors, what must it be?)

Vector Calculus Facts from cheatsheet:

Derivative of a norm-squared: $\frac{\partial}{\partial x} x^T x = 2x^T$

Scalar-vector product rule: $\frac{\partial}{\partial x}(f(x)g(x)) = f(x)\frac{\partial}{\partial x}(g(x)) + g(x)\frac{\partial}{\partial x}(f(x))$

This plus scalar calculus should suffice to get the answer.

14. Deja Vu: Denoising diffusion models for generation (28 pts)

In lecture, we talked about denoising diffusion models to get samples from a continuous distribution. This problem is about the potentially simpler binary case. We will assume that we have an unknown distribution of black-and-white images $P(x)$ together with a very large number of example images $\{x_1, x_2, \ldots, x_n\}$. Formally, each image can be viewed as a binary vector of length m, i.e. $x \in \{-1, +1\}^m$.

The first conceptual step in setting up a diffusion model is to choose the easy-to-sample distribution that we want to have at the end of the forward diffusion. For this, we choose m iid fair coin tosses (here we think of a fair coin as having a 50% chance of being +1 and a 50% chance of being -1) arranged into a vector.

Next, we need to choose a way to incrementally degrade the images. Let Y_0 start with whatever image sample x we want to start with. At diffusion stage t, we generate Y_t from Y_{t-1} by randomly flipping each pixel of Y_{t-1} independently with probability δ where δ is a small positive number.

It turns out that this process of rare pixel-flipping can be reinterpreted for easier analysis. For the j-th pixel at diffusion stage t, this process can alternatively be viewed as first flipping an independent coin $R_t[j]$ with a probability 2δ of coming up +1 and then, if $R_t[j] = +1$ replacing $Y_{t-1}[j]$ with a freshly drawn independent fair coin $F_t[j]$ that is equally likely to be -1 or +1. If $R_t[j] \neq +1$, we leave that pixel alone i.e. $Y_t[j] = Y_{t-1}[j]$.

(a) (4pts) We need to verify that if we do this and diffuse for sufficiently many stages T, that the resulting distribution is close to looking like m i.i.d. fair coins. **Show that the probability that pixel j has been replaced at some point by an independent fair coin by time T goes to 1 as $T \to \infty$.**

(HINT: It might be helpful to look at the probability that the desired event has not happened...)

(b) (6pts) To efficiently do diffusion training, we need a way to be able to quickly sample a realization of Y_t starting from $Y_0 = x_i$. **Give a procedure to sample a realization of Y_t given Y_0 without having to generate $Y_1, Y_2, \ldots, Y_{t-1}$.**

(HINT: Your answer should involve flipping at most two (potentially biased) coins for each pixel.)

(c) (2pts) For the reverse diffusion process that will be used during image generation and is being learned during training, our goal is to approximate $P(Y_{t-1}|Y_t)$ with a neural net that has learnable parameters θ. Suppose I give you a neural net whose input is a binary image Y_t and whose output is m real numbers that could each in principle be from $-\infty$ to $+\infty$ (for example, these could be the outputs of a linear layer). **Which of the following nonlinear activation functions would be most appropriate to convert a real number into a probability that we could use to sample whether the pixel in question should be a +1?**

◯ Sigmoid $\frac{1}{1+\exp(-x)}$

◯ ReLU $\max(0, x)$

◯ Tanh $\tanh(x) = \frac{\exp(2x)-1}{\exp(2x)+1}$

(d) (10pts) The goal of training is to approximate a probability distribution for random denoising. However, we do not actually have access to $P(Y_{t-1}|Y_t)$ and decide to use $P(Y_{t-1}|Y_t, Y_0)$ instead as a proxy. Note: this can be interpreted as finding an appropriate target distribution that corresponds to trying to predict Y_0 itself at this stage.

What is $P(Y_{t-1}[j] = +1|Y_t = y, Y_0 = x)$?

For simplicity, just do this calculation for the case $x[j] = +1$. To further help you save some time, you may use the following helper result that comes from Bayes' Rule. If A and B are both binary random variables where the prior probabilities are $P(A = +1) = \rho$ and $P(A = -1) = 1 - \rho$, with B being bit-flipped from A with independent probability δ — that is, $P(B = +1|A = +1) = 1 - \delta$, $P(B = -1|A = +1) = \delta$, $P(B = +1|A = -1) = \delta$, and $P(B = -1|A = -1) = 1 - \delta$ — then the conditional probabilities for A conditioned on B are given by:

$$P(A = +1|B = +1) = \frac{(1-\delta)\rho}{(1-\rho)\delta + (1-\delta)\rho} \tag{10}$$

$$P(A = -1|B = +1) = \frac{(1-\rho)\delta}{(1-\rho)\delta + (1-\delta)\rho} \tag{11}$$

$$P(A = +1|B = -1) = \frac{\delta\rho}{\rho\delta + (1-\delta)(1-\rho)} \tag{12}$$

$$P(A = -1|B = -1) = \frac{(1-\delta)(1-\rho)}{\rho\delta + (1-\delta)(1-\rho)} \tag{13}$$

(HINT: What is the distribution for $Y_{t-1}[j]$ given $Y_0[j]$?)

(e) (6pts) Let the (conditional) probability distribution (on whether each pixel is +1) output by our neural net with nonlinearity be $Q_t(Y_t)$. For training the denoising diffusion model $q(Y_{t-1}|Y_t)$, we choose to use SGD loss $D_{\mathrm{KL}}(P(Y_{t-1}|Y_t, Y_0 = x_i)\|Q_t(Y_t))$ where x_i is the random training image drawn, t is the random time drawn from 1 to T, and Y_t is the randomly sampled realization of the forward diffusion at time t starting with the image x_i at time 0. This ends up being a loss on the vector of probabilities coming out of $Q_t(Y_t)$ that can be written as a sum over the m entries in the vector of probabilities.

Given what you know about KL Divergence, what does this loss penalize most strongly? What does this loss look like at $t = 1$ in particular?

Because there are no cheat-sheets allowed and the order matters, we give you the formula for KL Divergence in the discrete case here: $D_{\mathrm{KL}}(P(z)\|Q(z)) = \sum_z P(z) \ln \frac{P(z)}{Q(z)}$.